

Numerical Simulation Laboratory

LECTURE 11

MACHINE LEARNING



1

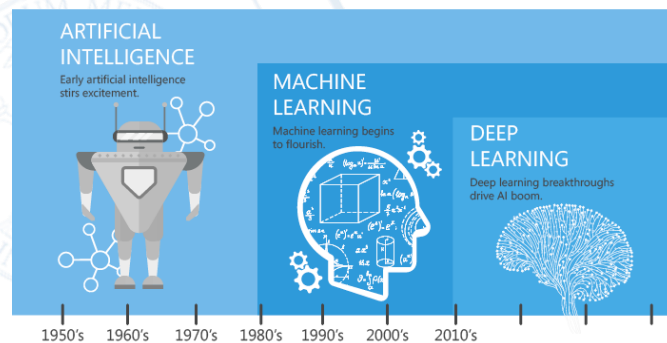
Outline

2

- Introduction to Machine Learning
- Introduction to feed-forward Neural Networks for supervised learning

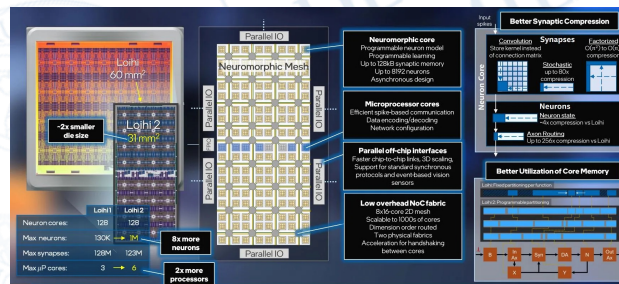


"Does your car have any idea why my car pulled it over?"



2

- Computers can calculate complex numerical calculations in a blink of an eye.
- In principle, they even have bigger processing power than human brain: 10^{10} transistors (but many more for a High Performance Parallel computer) with a switching time of 10^{-9} seconds against 9×10^{10} neurons in the whole nervous system (African elephant: 2.6×10^{11} ; human cerebral cortex: 1.6×10^{10} ; dolphin cerebral cortex: 3.7×10^{10}), with a switching time of about 10^{-3} seconds.
- Still, **brains outperform computers** in many aspects ... for sure from an energetic consumption point of view



- Another aspect is **learning and making decisions based on knowledge/experience**

3

3

Learning Machine Learning

4

- **Machine Learning (ML)**, data science, and statistics are fields that describe how to learn from, and make predictions about, data. The **availability of big datasets** is a hallmark of **modern science**, including physics, where data analysis has become an important component of diverse areas, such as experimental particle physics, observational astronomy and cosmology, condensed matter physics, biophysics ...
- Moreover, ML and data science are playing increasingly important roles in many aspects of modern technology ... therefore, having a deep understanding of the concepts and tools used in ML is an important skill that is increasingly relevant **in & out Physics** ... so let's start with learning machine learning!
- **ML is a subfield of AI with the goal of developing algorithms capable of learning from data automatically.** The basic premise of ML is to build algorithms that can receive input data and use statistical analysis to predict an output while updating outputs as new data becomes available.
- The last three decades have seen an unprecedented increase in our ability to generate and analyze large data sets.

4

Why study Machine Learning?

5

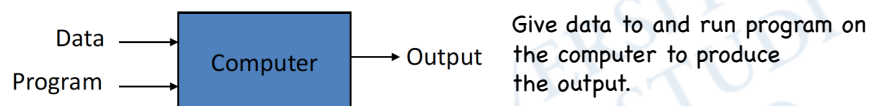
- This “big data” revolution has been induced by an exponential increase in computing power and memory (Moore’s law).
- Computations that were unthinkable a few decades ago can now be routinely performed on laptops. Specialized computing machines (such as GPU-based machines) are continuing this trend towards cheap, large-scale computation, suggesting that the “big data” revolution is here to stay.
- This increase in our computational ability has been accompanied by new techniques for analyzing and learning from large datasets. These techniques draw heavily from ideas in statistics, computational neuroscience, computer science, and physics.
- Physicists are uniquely situated to benefit from and contribute to ML. Many of the core concepts and techniques used in ML have their origins in Physics.
- Physicists have also been at the forefront of using “big data” and researchers are increasingly incorporating recent advances in ML and data science, and this trend is likely to accelerate in the future.

5

Types of Machine Learning

6

Traditional Programming



- ML can be divided into three broad categories:
 1. Supervised Learning
 2. Unsupervised Learning
 3. Reinforcement Learning
- While useful, the distinction between the three types of ML is sometimes fuzzy and fluid, and many applications often combine them in novel and interesting ways.
- For example, the recent success of Google DeepMind in developing ML algorithms that excel at tasks such as playing Go (<https://doi.org/10.1038/nature16961>) and video games employ deep reinforcement learning, combining reinforcement learning with supervised learning methods based on deep neural networks.

6

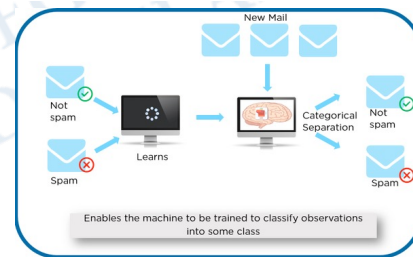
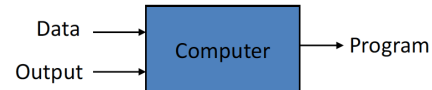
Supervised Learning

- **Supervised learning** concerns learning from labeled data, each data is tagged with the correct label (for example, a collection of pictures labeled as containing a cat or not containing a cat).
- The goal is to approximate the mapping function so well that when you have new input data (x) that you can predict the output variables (Y) for that data.
- Common supervised learning tasks include **classification** (A classification problem is when the output variable is a category, such as "red" or "blue" or "disease" and "no disease") and **regression** (the output variable is a real value, e.g. the value of a fitted function)

Traditional Programming

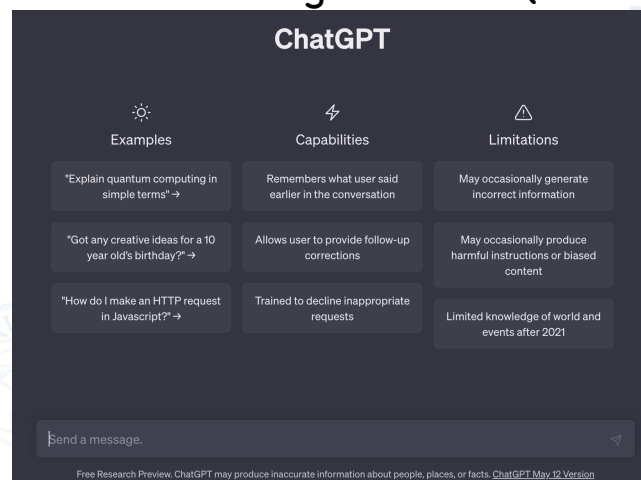


Machine Learning (supervised)



7

Supervised Learning in action (not only)!



- Overall, GPT-3.5 relies on deep learning techniques, including the Transformer architecture, self-attention mechanisms, and unsupervised pretraining followed by supervised fine-tuning, to generate its responses and provide conversational capabilities.

8

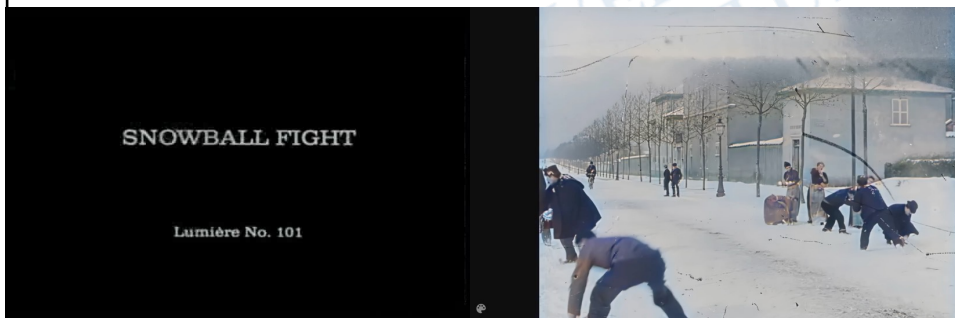
8

Other examples



9

Other examples



10

10

Unsupervised Learning

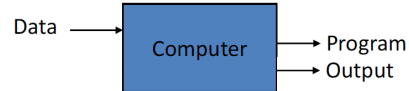
11

- **Unsupervised learning** is concerned with finding patterns and structure in unlabeled data.

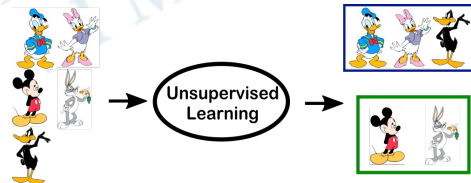
Traditional Programming



Machine Learning (unsupervised)



- Examples of unsupervised learning include **clustering** (you want to discover the inherent groupings in the data, such as grouping customers by purchasing behavior), **association** (you want to discover rules that describe large portions of your data, such as people that buy X also tend to buy Y), **generative modeling**, etc.

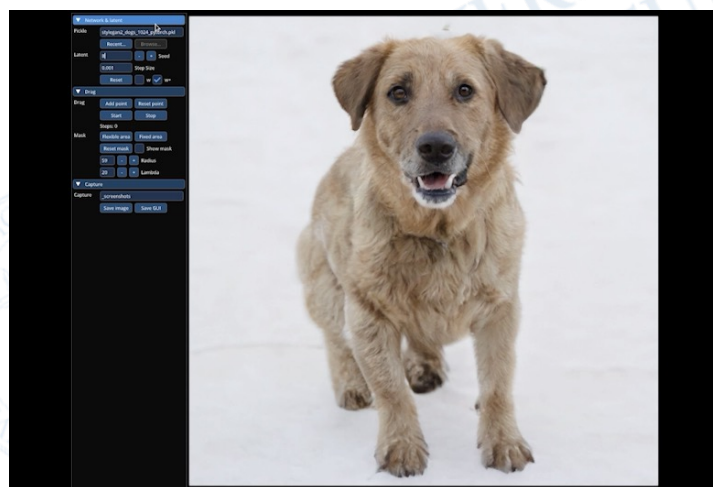


- In our training data, we don't provide any label to the corresponding data. **The unsupervised model is able to separate both the characters by looking at the type of data and models the underlying structure or distribution in the data** in order to learn more about it.

11

Unsupervised Learning in action!

- Example of sophisticated generative modelling: generative adversarial networks (GANs) <https://vcai.mpi-inf.mpg.de/projects/DragGAN/>



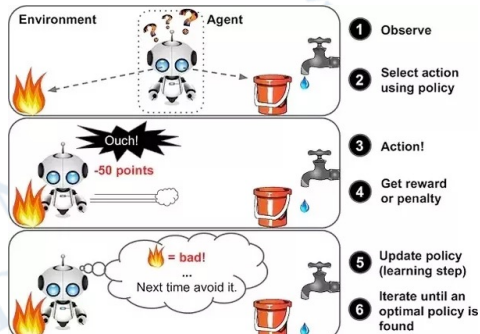
12

12

Reinforcement Learning

13

- Finally, a **reinforcement learning** algorithm, or agent, **learns by interacting with its environment**. The agent receives rewards by performing correctly and penalties for performing incorrectly.



- The agent **learns without intervention from a human** by maximizing its reward and minimizing its penalty.
- Reinforcement learning is based on a mathematical framework that combines elements of dynamic programming, control theory, and **Markov decision processes** (MDPs). An MDP consists of a set of states, actions, transition probabilities, rewards, and a discount factor. The system being modeled is assumed to satisfy the Markov property, which means the future state depends only on the current state and action, and not on the past history.

13

Reinforcement Learning in action!

- In the following application of reinforcement learning agent has only two possible moves/actions: **right** or **left**
- It (or he?) can adjust also the rapidity of the moves



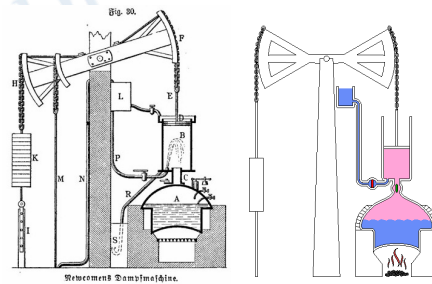
14

14

ML: 'pre-scientific' technology

15

- Despite these amazing successes for Deep Learning we are still in a 'pre-scientific' era
- We know how to build artificially intelligent systems and algorithms but we are not yet able to explain well why one algorithm is more efficient than another, the 'thermodynamics of learning' is missing
- Similar to the era of the First Industrial Revolution (1760-1820): development and application of steam engines without understanding the principles that explain their efficiency, Thermodynamics:
 - 1820: Carnot's Theorem
 - 1848: 2nd principle of thermodynamics (Kelvin)
 - 1850: Heat-Energy Equivalence (Joule)
 - 1855: Entropy (Clausius)
 - 1876: Thermodynamic transformations (Gibbs)



15

Setting up a problem in supervised ML

16

Many problems in supervised ML and data science starts with the same ingredients:

- Input data
- Cause
- Initial conditions, boundary conditions



- Output data
- Effect
- Final conditions, final state

$f(X;p)$: Model/map/function/functional
Causal relations/connections

- Theory/parameters

- The **first ingredient** is the **dataset** $D = (X; Y)$ where X is a set of independent variables and Y is a set of dependent variables.
- The **second** is the **model** $f(X;p)$, which is a function $f: x \rightarrow y$ of the **parameters** p . That is, f is a function used to predict an output from a vector of input variables.
- The **final ingredient** is the **cost function** $C[y; f(X;p)]$ that allows us to judge how well the model performs on the observations y . **The model is fit by finding the value of p that minimizes the cost function.** For example, one commonly used cost function is the squared error. Minimizing the squared error cost function is known as the method of least squares

16

Train & test

17

ML researchers and data scientists follow a **standard recipe** to obtain models that are useful for prediction problems.

- The **first step** in the analysis is to **randomly divide the dataset D into two mutually exclusive groups D_{train} and $D_{\text{test/validation}}$** called the training and test/validation sets. Performing some analysis (such as using the data to select important variables) before partitioning the data is a common pitfall that can lead to incorrect conclusions.

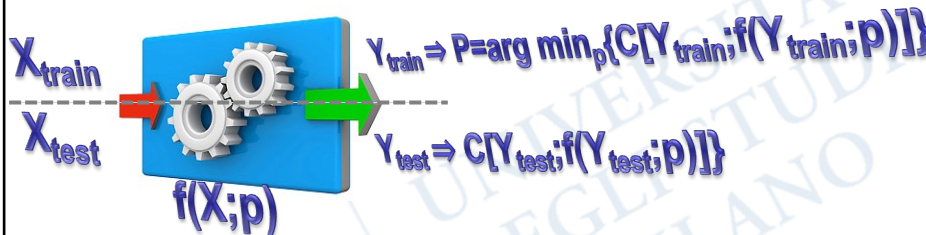


- Typically, the majority of the data are partitioned into the training set (e.g., 90%) with the remainder going into the test set.

17

- The model is **fit by minimizing the cost function using only the data in the training set** $P = \arg \min_p \{C[Y_{\text{train}}; f(X_{\text{train}}; p)]\}$

18



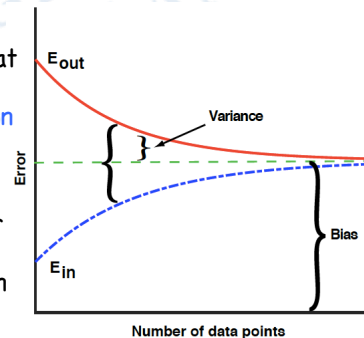
- Finally, the **performance of the model is evaluated** by computing the cost function using the test set $C[Y_{\text{test}}; f(X_{\text{test}}; p)]$
- The value of the cost function for the best fit model on the training set is called the **in-sample (or training) error** $E_{\text{in}} = C[Y_{\text{train}}; f(X_{\text{train}}; p)]$ and the value of the cost function on the test set is called the **out-of-sample (or generalization) error** $E_{\text{out}} = C[Y_{\text{test}}; f(X_{\text{test}}; p)]$
- Comparing E_{in} and E_{out} is known as the cross-validation procedure in ML literature.** One of the most important observations we can make is that $E_{\text{out}} \geq E_{\text{in}}$ **almost always**. This is because the test data was now used in the fitting procedure.

18

Statistical Learning Theory

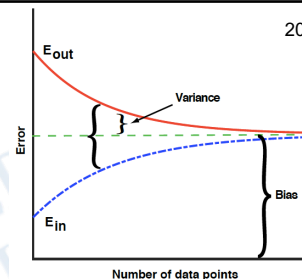
19

- This raises a natural question: Can we say something general about the relationship between E_{in} and E_{out} ? This is the domain of **statistical learning theory**, and we give a brief overview of the main results in this section.
- The basic intuitions of statistical learning can be summarized in two simple observations. The **first one**, shown in the following figure, shows the typical out-of-sample error, E_{out} , and in-sample error, E_{in} , as a function of the amount of training data.
- In making this graph, we have assumed that the **true data is drawn from a complicated distribution, so that we cannot exactly learn $f(x)$** .
- Hence, after a quick initial drop (not shown in figure), E_{in} will increase with the number of data points, because our models are not powerful enough to learn the true function we are seeking to approximate.
- In contrast, E_{out} will decrease with the number of data points.



19

- As the number of data points gets large, the training data set becomes more representative of the true distribution from which the data is drawn.
- For this reason, in the infinite data limit, E_{in} and E_{out} must approach the same value, which is called the "bias" of our model.
- In general, the **more complex the model class** we use, **the smaller the bias**. However, we do not generally have an infinite amount of data. For this reason, we cannot know the bias and we have to minimize E_{out} .
- E_{out} can be naturally decomposed into a **bias**, which measures how well we can hypothetically do in the infinite data limit, and a **"variance"**, which measures the typical errors introduced in training our model due to having a finite training set.
- The final quantity shown in figure is the **difference between the generalization and training error**. It measures how much worse we would do on a new data set compared to our training data. **Models with a large difference** between the in-sample and out-of-sample errors are said to **"overfit" the data**.

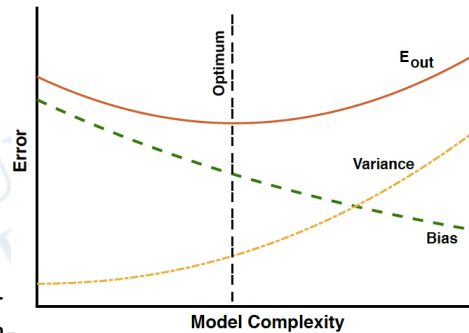


20

- One of the lessons of statistical learning theory is that it is not enough to simply minimize the training error, because the out-of-sample error can still be large.

21

- The second observation shows E_{out} as a function of "model complexity" (which is related to the number of parameters we are using to approximate the true function $f(x)$)



- If we consider a training dataset of a fixed size, E_{out} will be a non-monotonic function of the model complexity, and is generally minimized for models with intermediate complexity
- In fact, even though using a more complicated model always reduces the bias, at some point the model becomes too complex for the amount of training data and the generalization error becomes large due to high variance.

21

Key Elements of Machine Learning

22

All machine learning algorithms are combinations of three components:

- Representation:** how to represent knowledge. Examples include decision trees, graphical models, neural networks, support vector machines and others.
- Evaluation:** the way to evaluate candidate programs (hypotheses). Examples include accuracy, squared error, likelihood, posterior probability, cost, entropy, KL divergence and others.
- Optimization:** the way candidate programs are generated, known as the search process. For example, stochastic gradient descent, quasi-Newton methods, convex optimization, constrained optimization, etc.

Representation: the *landscape* of possible models, the playing field allowed by a given representation.

Evaluation: Preferences over the landscape

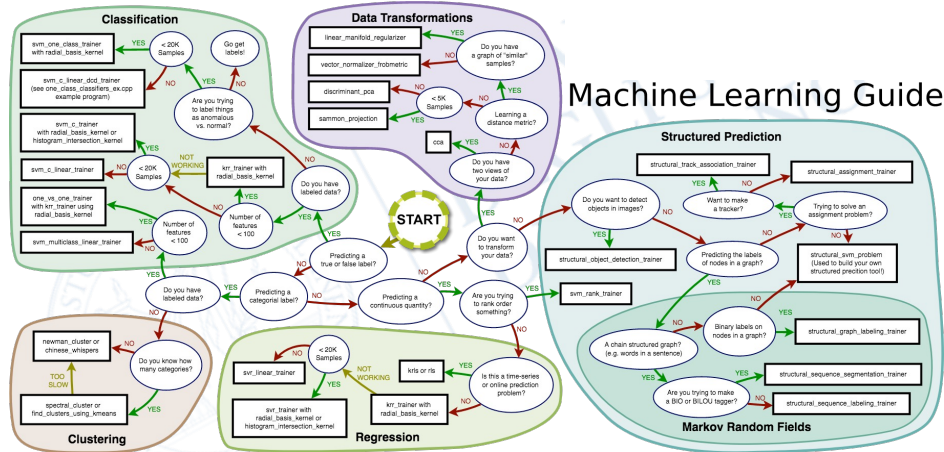
Optimization: Strategy for fulfilling preferences

22

Machine Learning is huge!

23

The picture above is just to show that ML researcher over the years have developed a large set of techniques to solve specific problems to learn from data ...



We will go on introducing one of the most powerful and widely used algorithms when it comes to the subfield of ML called **deep learning**

23

An introduction to feedforward Neural Networks

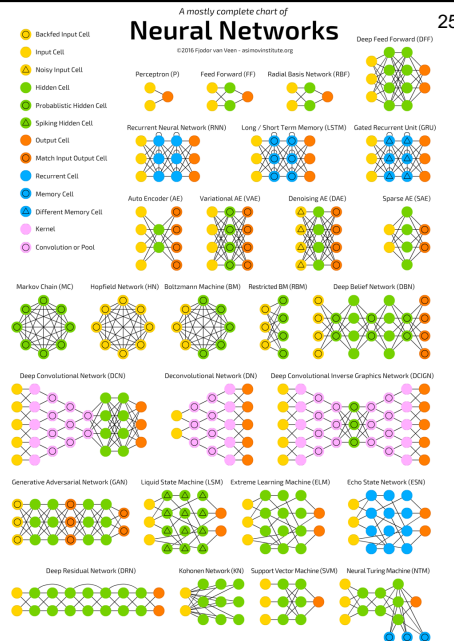
24

- Over the last decade, **Neural Networks (NNs)** have emerged as the one of most powerful and widely-used supervised learning techniques.
- NNs have a long history, but re-emerged to prominence after a rebranding as "DeepLearning" and **Deep Neural Networks (DNNs)** in the mid 2000s.
- DNNs truly caught the attention of the wider machine learning community and industry in 2012 when a group of data scientists used a GPU-based DNN model (AlexNet) to lower the error rate on the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) by an incredible twelve percent from 28% to 16%. Just three years later, a machine learning group from Microsoft achieved an error of 3.57% using an ultradeep residual neural network (ResNet) with 152 layers!
- Since then, DNNs have become the workhorse technique for many image and speech recognition based machine learning tasks. The large-scale industrial deployment of DNNs has given rise to a number of **high-level libraries and packages** (Caffe, Keras, Pytorch, and TensorFlow) that make it easy to **quickly code and deploy DNNs**.

24

NNs categories

- Conceptually, it is helpful to divide neural networks into four categories:
 - general purpose neural networks for supervised learning,
 - neural networks designed specifically for image processing, the most prominent example of this class being Convolutional Neural Networks (CNNs)
 - neural networks for sequential data such as Recurrent Neural Networks (RNNs), and
 - neural networks for unsupervised learning such as Deep Boltzmann Machines.

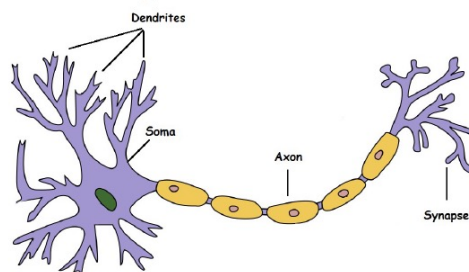


In physics, DNNs and CNNs have already found numerous applications; see: "Machine learning and the physical sciences" (<https://doi.org/10.1103/RevModPhys.91.045002>)

25

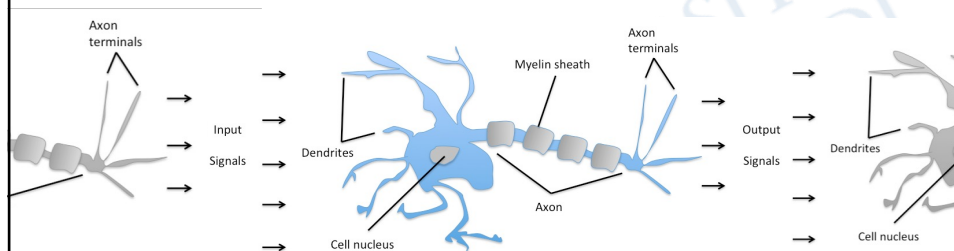
Biology behind the idea

- So, what nature's concepts are Neural Networks trying to imitate?
- As you are probably aware, the smallest unit of the nervous system is a neuron. These are cells with similar and simple structure. Yet, by continuous communication, these cells achieve enormous processing power.
- If you put it in simple terms, neurons are just switches. These switches generate an output signal if they receive a certain amount of input stimuli. This output signal is input for another neuron.
- Each neuron has these components: Body (also known as soma), Dendrites, Axon
- Body (soma) of a neuron carries out the basic life processes of a neuron. Every neuron has a single axon. This is a long part of the cell; it acts like a wire and it is an output of the neuron.



26

- Dendrites, on the other hand, are inputs of neuron and each neuron has multiple dendrites. These inputs and outputs, dendrites and axons, of different neurons come close via **axon terminals**: the **synapses**



Schematic of a biological neuron.

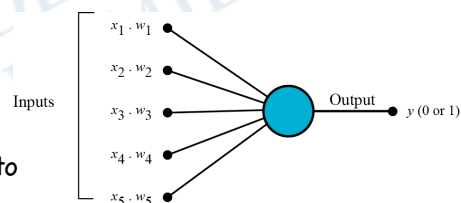
- Through the synapses signals are carried by **neurotransmitter molecules**, and each serves a different type of neuron (e.g. the famous serotonin and dopamine).
- The amount and type of these chemicals will dictate how "strong" the input to the neuron is. And, if there is enough input on all dendrites, the soma will "fire up" the signal on the axon, and transmit it to the next neuron.

27

Perceptrons

28

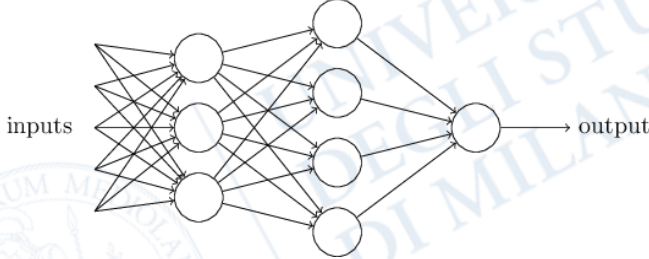
- What is a neural network? To get started, I'll explain a type of artificial neuron called a **perceptron**. Perceptrons were developed in the 1950s and 1960s by the scientist Frank Rosenblatt, inspired by earlier work by Warren McCulloch and Walter Pitts.
- Today, it's more common to use other models of artificial neurons - in much modern work on neural networks, the main neuron model used is one called the **sigmoid neuron**. We'll get to sigmoid neurons shortly. But to understand why sigmoid neurons are defined the way they are, it's worth taking the time to first understand perceptrons.
- So how do perceptrons work? **A perceptron takes several binary inputs, $x_1, x_2, \dots$, and produces a single binary output:**
- Rosenblatt proposed a simple rule to compute the output. He introduced **weights, $w_1, w_2, \dots$** , real numbers expressing the importance of the respective inputs to the output. **The output, 0 or 1, is determined by whether the weighted sum $\sum_j w_j x_j$ is less than or greater than some threshold value.** Just like the weights, the threshold is a real number which is a parameter of the neuron.



28

29

- A way you can think about the perceptron is that it's a device that makes decisions by weighing up evidence/data.
- Obviously, the perceptron isn't a complete model of human decision-making! But it should seem plausible that a complex network of perceptrons could make quite subtle decisions:



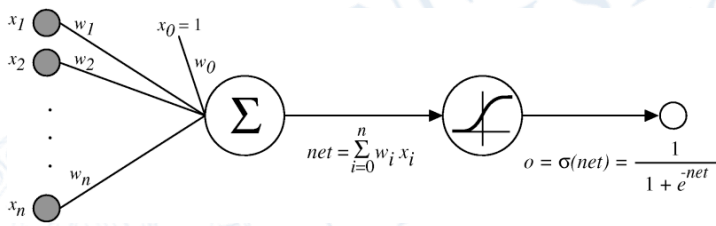
- There is however problem when our network contains perceptrons. In fact, a small change in the weights or threshold of any single perceptron in the network can sometimes cause the output of that perceptron to completely flip, say from 0 to 1. That flip may then cause the behaviour of the rest of the network to completely change in some very complicated way
- That makes it difficult to see how to gradually modify the weights and biases so that the network gets closer to the desired behaviour.

29

Sigmoid neurons

30

- We can overcome this problem by introducing a new type of artificial neuron called a **sigmoid neuron**. Sigmoid neurons are similar to perceptrons, but modified so that small changes in their weights and bias cause only a small change in their output. That's the crucial fact which will allow a network of sigmoid neurons to learn.



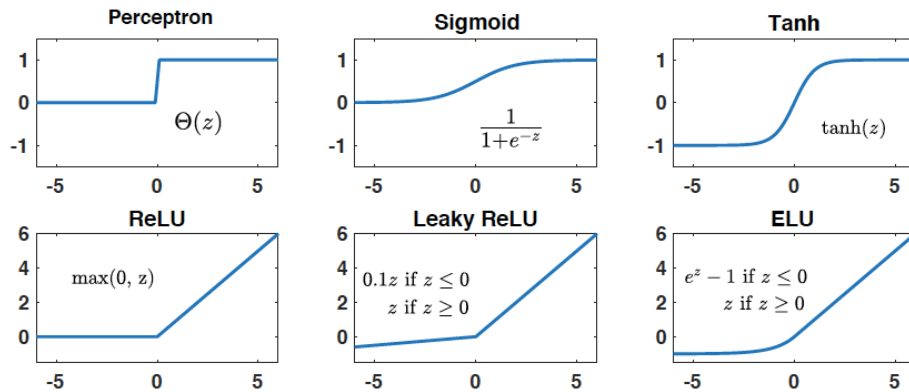
- Just like a perceptron, the sigmoid neuron has inputs, $x_1, x_2, \dots$. But the output is not 0 or 1. Instead, it's $\sigma(z) = 1/(1 + \exp(-z))$, where σ is called the sigmoid function, $z = \mathbf{w} \cdot \mathbf{x} + w_0$ and w_0 (or b) the bias.
- Note that It's only when $\mathbf{w} \cdot \mathbf{x} + b$ is of modest size that there's much deviation from the perceptron model. The exact form of σ isn't so important - what really matters is the shape of the function.

30

Activation functions

31

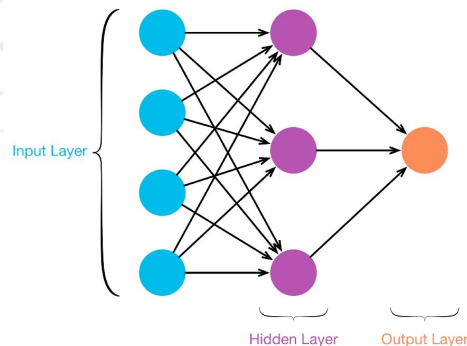
- Various possible non-linear activation functions for neurons:



- In modern DNNs, it has become common to use non-linear functions that do **not saturate for large inputs (bottom row)** rather than **saturating functions (top row)**.

31

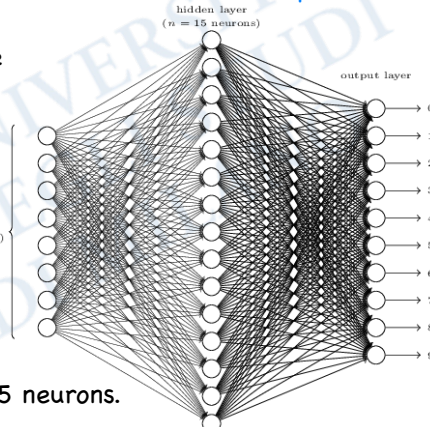
- If σ had in fact been a step function, then the sigmoid neuron would be a perceptron, since the output would be 1 or 0 depending on whether $\mathbf{w} \cdot \mathbf{x} + w_0$ was positive or negative.
- It's **the smoothness of the σ function that is the crucial fact**, not its detailed form. The smoothness of σ means that **small changes Δw_j in the weights and Δw_0 in the bias will produce a small change Δ_{out} in the output from the neuron.**
- The basic idea of all neural networks is to layer neurons in a hierarchical fashion; suppose to have a complex network obtained combining more sigmoid neurons:
- The **leftmost layer** in this network is called the **input layer**, and the neurons within the layer are called input neurons. The rightmost or **output layer** contains the output neurons, or, as in this case, a single output neuron. The **middle layer** is called a **hidden layer**.
- The design of the input and output layers in a network is often straightforward because they should adapt to the **dataset $D = (X; Y)$** .



32

33

- For example, suppose we're trying to determine the digit in a handwritten image. A natural way to design the network is to encode the intensities of the image pixels into the input neurons. If the image is a 28 by 28 greyscale image, then we'd have $784=28 \times 28$ input neurons
- For simplicity I've omitted most of the 784 input neurons in the diagram. The input pixels are greyscale, with a value of 0.0 representing white, a value of 1.0 representing black (or the opposite!), and in between values representing gradually darkening shades of grey.
- The second layer of the network is a hidden layer. The example shown illustrates a small hidden layer, containing just $n=15$ neurons.
- The output layer of the network contains 10 neurons. We number the output neurons from 0 through 9, and figure out which neuron has the highest activation value. If that neuron is, say, neuron number 6, then our network will guess that the input digit was a 6. And so on for the other output neurons.



33

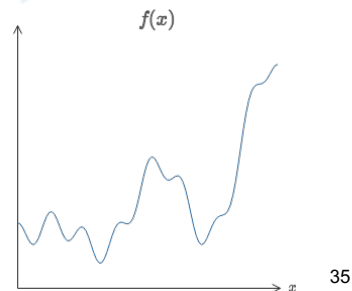
Universal approximation theorems

- Thus, the whole neural network can be thought of as a complicated nonlinear transformation of the inputs X into an output Y that depends on the weights and biases of all the neurons in the input, hidden, and output layers.
- The use of hidden layers greatly expands the representational power of a NN. Perhaps, the most formal expression of the increased representational power of neural networks is the universal approximation theorem which states that a neural network with a single hidden layer can approximate any continuous, multi-input/multi-output function with arbitrary accuracy (G. Cybenko, 1989, for sigmoid; Kurt Hornik, 1991, etc. etc.)
- Although feed-forward networks with a single hidden layer are universal approximators, the width of such networks has to be exponentially large
- In 2017 Lu et al. proved universal approximation theorem for width-bounded deep neural networks. In particular, they showed that width- $n+4$ networks with ReLU activation functions can approximate any Lebesgue integrable function on n -dimensional input space if the depth of the network is allowed to grow

34

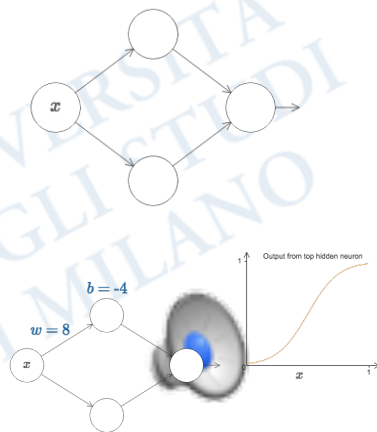
34

- These results tell us that **neural networks have a kind of universality**. No matter what function we want to compute, we know that there is a neural network which can do the job.
- One can give a simple and mostly **visual explanation** of the universality theorem
- The basic idea is that **hidden neurons allow neural networks to generate step functions (characteristic functions) with arbitrary offsets and heights**. These can then be **added together to approximate arbitrary functions**.
- The proof also makes clear that the more complicated a function, the more hidden units (and free parameters) are needed to approximate it. Hence, **the applicability of the approximation theorem to practical situations should not be overemphasized**
- To understand why the universality theorem is true, **let's start** by understanding how to construct a neural network which approximates a function **with just one input and one output ...**



35

- Once we've understood this special case it's actually pretty easy to extend to functions with many inputs and many outputs.
- To build insight into how to construct a network to compute f , **let's start with a network containing just a single hidden layer, with two hidden neurons, and an output layer** containing a single output neuron:
- To get a feel for how components in the network work, **let's focus on the top hidden neuron**. What's being computed by the hidden neuron is $\sigma(wx+b)$, where $\sigma(z) \equiv 1/(1+e^{-z})$ is the sigmoid function.
- By **increasing the weight, w , the curve gets steeper**, until eventually it begins to look like a **step function**. On the contrary, by decreasing the weight, the curve broadens out
- As the **bias decreases the graph moves to the right**, but its shape doesn't change; as the bias increases the graph moves to the left, but, again, its shape doesn't change.



36

37

- It's actually quite a bit easier to work with step functions than general sigmoid functions. The reason is that in the output layer we add up contributions from all the hidden neurons. It's easy to analyze the sum of a bunch of step functions, but rather more difficult to reason about what happens when you add up a bunch of sigmoid shaped curves. And so it makes things much easier to assume that our hidden neurons are outputting step functions
- We do this by fixing the weight w to be some very large value. It will greatly simplify our lives to describe hidden neurons using just a single parameter, s , which is the step position, $s = -b/w$
- Let's take a look at the behavior of the entire network. In particular, we'll suppose the hidden neurons are computing step functions parameterized by step points s_1 (top neuron) and s_2 (bottom neuron). And they'll have output weights w_1 and w_2

37

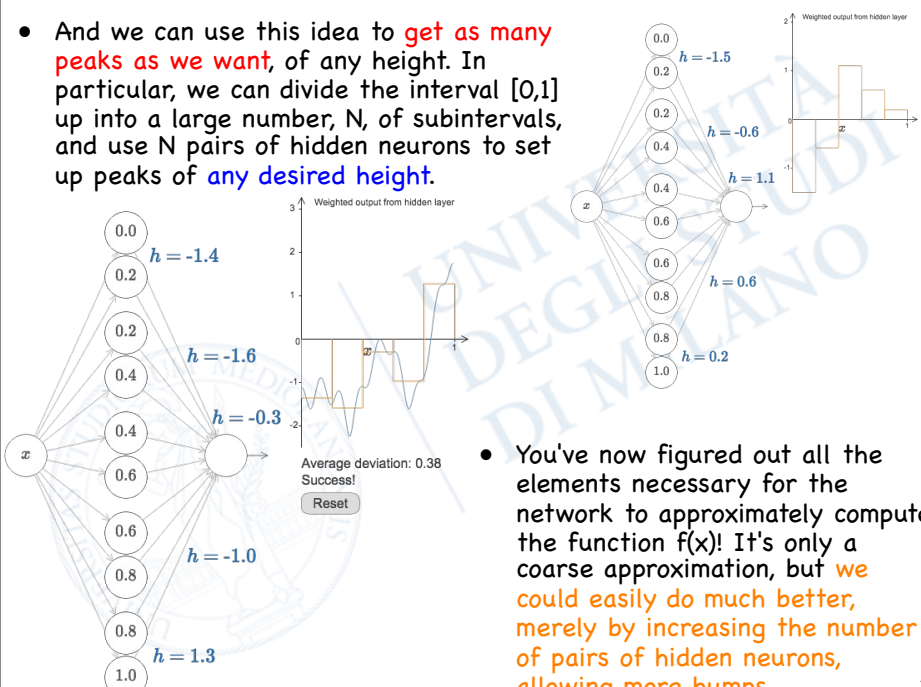
38

- Finally, setting w_1 to be positive and $w_2 = -w_1$ negative, you can get a "bump" function, which starts at point s_1 , ends at point s_2 , and has height h . For instance, the weighted output might look like this:
- Of course, we can rescale the bump to have any height at all. Let's use a single parameter, h , to denote the height.
- We can use our bump-making trick to get two bumps, by gluing two pairs of hidden neurons together into the same network:

38

38

- And we can use this idea to **get as many peaks as we want**, of any height. In particular, we can divide the interval $[0,1]$ up into a large number, N , of subintervals, and use N pairs of hidden neurons to set up peaks of **any desired height**.

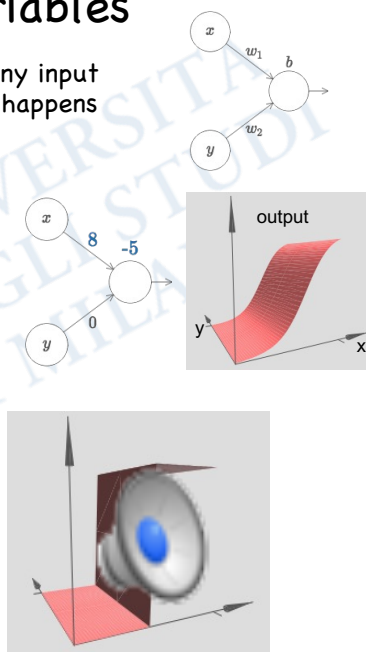


- You've now figured out all the elements necessary for the network to approximately compute the function $f(x)$! It's only a coarse approximation, but **we could easily do much better, merely by increasing the number of pairs of hidden neurons, allowing more bumps.**

39

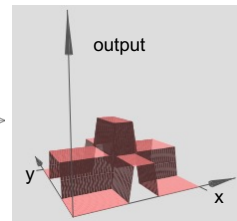
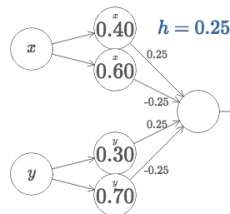
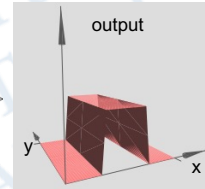
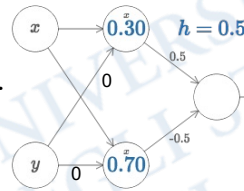
Many input variables

- Let's extend our results to the case of many input variables. We'll start by considering what happens when we have **two inputs to a neuron**:
- As you can see, with $w_2=0$ the input y makes no difference to the output from the neuron. It's as though x is the only input.
- Given this, what do you think happens when we increase the weight w_1 , with $w_2=0$? Just as in our earlier discussion, **as the input weight gets larger the output approaches a step function**. The difference is that now the step function is in 3D. Also as before, **we can move the location of the step point around by modifying the bias**. The actual location of the step point is $s_x = -b/w_1$.



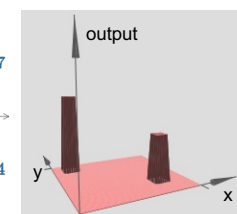
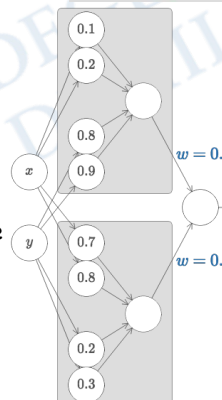
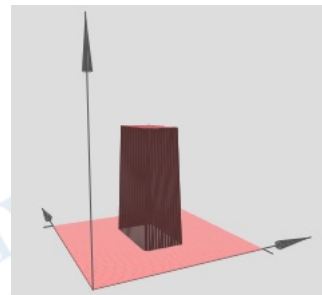
40

- Of course, it's also possible to get a step function in the y direction, by making the weight on the y input very large, and the weight on the x equal to 0
- We can use the step functions we've just constructed to compute a **three-dimensional bump function**. To do this, we use two neurons, each computing a step function in the x direction. Then we combine those step functions with weight h and $-h$, respectively, where h is the desired height of the bump.
- Of course, we can easily make a bump function in the y direction, by using two step functions in the y direction
- Let's consider what happens when we **add up two bump functions**, one in the x direction, the other in the y direction, both of height h (To simplify the diagram I've dropped the connections with zero weight.):



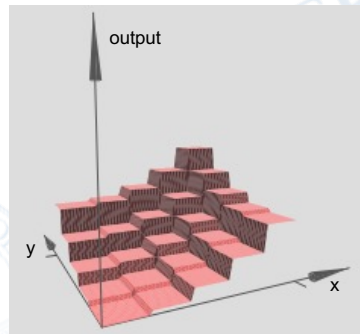
41

- If we choose the threshold appropriately - say, a value of $3h/2$, which is sandwiched between the height of the plateau and the height of the central tower - **by adjusting the bias we could squash the plateau down to zero, and leave just the tower standing.**
- Let's try gluing two such networks together, in order to compute two different tower functions. To make the respective roles of the two sub-networks clear I've put them in separate boxes, below: each box computes a tower function, using the technique described above. The graph on the right shows the weighted output from the second hidden layer, that is, it's a **weighted combination of tower functions.**



42

- The same idea can be used to compute as many towers as we like.
- We can also **make them as thin as we like, and whatever height we like**. As a result, we can ensure that the weighted output from the second hidden layer **approximates any desired function of two variables**:



- By using more input neurons and adjusting the complexity of the NN you can approximate functions of many variables
- If you want to practice directly yourself with step functions, go to: <http://neuralnetworksanddeeplearning.com>
Especially, see Chapter 4

43

- Modern neural networks generally contain **multiple hidden layers** (hence the “deep” in **deep learning**).
- Increasing the number of layers increases the number of parameters and hence the representational power of neural networks.
- **Choosing the exact network architecture for a neural network remains an art that requires extensive numerical experimentation and intuition, and is often times problem-specific.** Both the number of hidden layers and the number of neurons in each layer can affect the performance of a neural network. There seems to be no single recipe for the right architecture for a neural net that works best.
- **Anyway, the number of parameters in the neural net should be large enough to prevent underfitting and not too large to prevent overfitting**

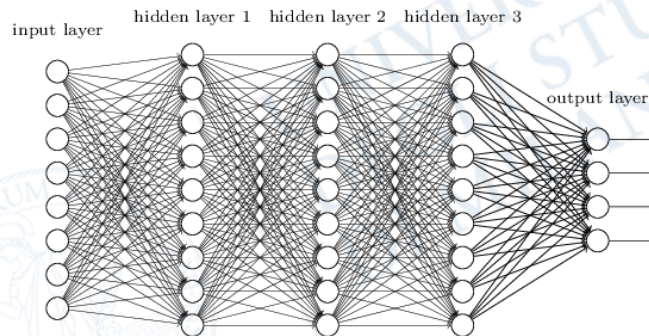


44

Training deep networks

45

Neural networks differ from simpler supervised procedures in that generally they contain multiple hidden layers that make taking the gradient computationally more difficult.



How can we fit a model with a NN & DNN? The basic procedure for training neural nets is:

- construct a cost/loss function and then use some gradient descent technique to minimize the cost function and find the optimal weights and biases.

45

The cost/loss function

46

- Recall that in most cases, the top output layer of our neural net is either a continuous predictor or a classifier that makes discrete (categorical) predictions. Depending on whether one wants to make continuous or categorical predictions, one must utilize a different kind of loss function
- For continuous data, the loss functions that are commonly used include the mean squared error

$$E(\vec{w}) = \frac{1}{n} \sum_{i=1}^n |\bar{Y}_i - f(\bar{Y}_i; \vec{w})|^2$$

- And the mean absolute error

$$E(\vec{w}) = \frac{1}{n} \sum_{i=1}^n |\bar{Y}_i - f(\bar{Y}_i; \vec{w})|$$

- The full cost function often includes additional terms that implement some sort of regularization
- For categorical data, when the output layer is taken to be a classifier for binary data with only two types of labels (or a softmax classifier if there are more than two types of labels), the most commonly used loss function is the cross-entropy.

46

- The **cross-entropy** between the true labels $Y_i \in \{0;1\}$ and the predictions is given by

$$E(\bar{\mathbf{w}}) = -\sum_{i=1}^n Y_i \log f(Y_i; \bar{\mathbf{w}}) + (1 - Y_i) \log [1 - f(Y_i; \bar{\mathbf{w}})]$$

- Note, in fact, that if the weights $\mathbf{w}$ predict the correct labeling of the data, for example $f(Y_i; \mathbf{w}) \approx 0$ when $Y_i=0 \forall i$ except when $i=k$, where $Y_k=1$ and $f(Y_k; \mathbf{w}) \approx 1$, the cross-entropy is

$$E(\bar{\mathbf{w}}) \approx -Y_k \log f(Y_k; \bar{\mathbf{w}}) - \sum_{i=1(\neq k)}^n \log [1 - f(Y_i; \bar{\mathbf{w}})] \approx 0$$

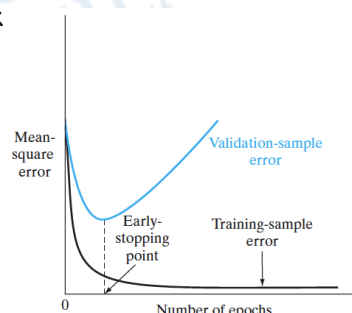
otherwise $E(\mathbf{w}) > 0$

- Having defined an architecture and a cost function, **we must now train the model**. Similar to other supervised learning methods, we make use of **gradient descent-based methods** to optimize the cost function.
- Depending on the architecture, data, and computational resources, different optimizers may work better on the problem, though **vanilla Stochastic Gradient Descent is a good first choice**.
- Finally, we note that calculating the gradients for a neural network requires a **specialized algorithm**, called **Backpropagation** which forms the heart of any neural network training procedure.

47

Early-Stopping Method of Training

- Ordinarily, a DNN trained with the back-propagation algorithm **learns in stages**, moving from the realization of fairly simple to more complex mapping functions as the training session progresses.
- This process is exemplified by the fact that in a typical situation, **the mean-square error decreases with an increasing number of epochs** used for training, and it is very **difficult to figure out when it is best to stop training** if we were to look at the learning curve for training all by itself
- In particular, it is possible for the network to end up **overfitting** the training data if the training session is not stopped at the right point.
- We may identify the onset of overfitting through the use of cross-validation by comparing E_{in} & E_{out}
- This heuristic suggests that the minimum point on the validation learning curve be used as a sensible criterion for **early-stopping** the training session



48

Lecture 11: Suggested books

- P. Mehta, C.H. Wang, A.G.R. Day, and C. Richardson, *A high-bias, low-variance introduction to Machine Learning for physicists*, <https://doi.org/10.1016/j.physrep.2019.03.001> (2019)
- M. Nielsen, *Neural Networks and Deep Learning*, <http://neuralnetworksanddeeplearning.com>, on-line book
- S. Haykin, *Neural Networks and Learning Machines*, Pearson (2009)
- S. Theodoridis, *Machine Learning. A Bayesian and Optimization Perspective*, Elsevier (2015)
- K.P. Murphy, *Machine Learning. A Probabilistic Perspective*, MIT press (2012)

49